

is subsequently trained from data using backpropagation. In [24], first-order logic programs in the form of Horn clauses are used to define a neural network that can solve Inductive Logic Programming tasks, starting from the most specific hypotheses covering the set of examples. Lifted relational neural networks [66] is a declarative framework where a Datalog program is used as a compact specification of a diverse range of existing advanced neural architectures, with a particular focus on Graph Neural Networks (GNNs) and their generalizations. In [56] a weighted Real Logic is introduced and used to specify neurons in a highly modular neural network that resembles a tree structure, whereby neurons with different activation functions are used to implement the different logic operators.

To some extent, it is also possible to specify neural architectures using logic in LTN. For example, a user can define a classifier $P(x, y)$ as the formula $P(x, y) = (Q(x, y) \wedge R(y)) \vee S(x, y)$. $\mathcal{G}(P)$ becomes a computational graph that combines the sub-architectures $\mathcal{G}(Q)$, $\mathcal{G}(R)$, and $\mathcal{G}(S)$ according to the syntax of the logical formula.

5.3. Neurosymbolic architectures for the integration of inductive learning and deductive reasoning

These architectures seek to enable the integration of inductive and deductive reasoning in a unique fully differentiable framework [15, 23, 41, 46, 47]. The systems that belong to this class combine a neural component with a logical component. The former consists of one or more neural networks, the latter provides a set of algorithms for performing logical tasks such as model checking, satisfiability, and logical consequence. These two components are tightly integrated so that learning and inference in the neural component are influenced by reasoning in the logical component and vice versa. Logic Tensor Networks belong to this category. Neurosymbolic architectures for integrating learning and reasoning can be further separated into three sub-classes:

1. Approaches that introduce additional layers to the neural network to encode logical constraints which modify the predictions of the network. This sub-class includes Deep Logic Models [46] and Knowledge Enhanced Neural Networks [15].
2. Approaches that integrate logical knowledge as additional constraints in the objective function or loss function used to train the neural network (LTN and [23, 33, 47]).
3. Approaches that apply (differentiable) logical inference to compute the consequences of the predictions made by a set of base neural networks. Examples of this sub-class are DeepProbLog [41] and Abductive Learning [14].

In what follows, we revise recent neurosymbolic architectures in the same class as LTN: *Integrating learning and reasoning*.

Systems that modify the predictions of a base neural network: Among the approaches that modify the predictions of the neural network using logical constraints are Deep Logic Models [46] and Knowledge Enhanced Neural Networks [15]. Deep Logic Models (DLM) are a general architecture for learning with constraints. Here, we will consider the special case where constraints are expressed by logical formulas. In this case, a DLM predicts the truth-values of a set of n ground atoms of a domain $\Delta = \{a_1, \dots, a_k\}$. It consists of two models: a neural network $f(x \mid w)$ which takes as input the features x of the elements of Δ and produces as output an evaluation f for all the ground atoms, i.e. $f \in [0, 1]^n$, and a probability distribution $p(y \mid f, \lambda)$ which is modeled by an undirected graphical model of the exponential family with each logical constraint characterized by a clique that contains the ground atoms, rather similarly to GNNs. The model returns the assignment to

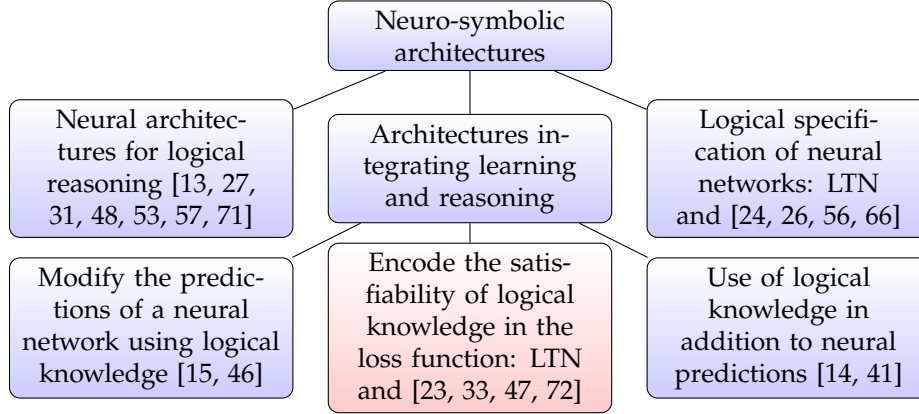


Figure 24: Three classes of neurosymbolic approaches with *Architectures Integrating Learning and Reasoning* further subdivided into three sub-classes, with LTN belonging to the sub-class highlighted in red.

the atoms that maximize the weighted truth-value of the constraints and minimize the difference between the prediction of the neural network and a target value $\mathbf{y}$. Formally:

$$\text{DLM}(\mathbf{x} \mid \boldsymbol{\lambda}, \mathbf{w}) = \underset{\mathbf{y}}{\operatorname{argmax}} \left(\sum_c \lambda_c \Phi_c(\mathbf{y}_c) - \frac{1}{2} \|\mathbf{y} - \mathbf{f}(\mathbf{x} \mid \mathbf{w})\|^2 \right)$$

Each $\Phi_c(\mathbf{y}_c)$ corresponds to a ground propositional formula which is evaluated w.r.t. the target truth assignment $\mathbf{y}$, and λ_c is the weight associated with formula Φ_c . Intuitively, the upper model (the undirected graphical model) should modify the prediction of the lower model (the neural network) minimally to satisfy the constraints. $\mathbf{f}$ and $\mathbf{y}$ are truth-values of all the ground atoms obtained from the constraints appearing in the upper model in the domain specified by the data input.

Similar to LTN, DLM evaluates constraints using fuzzy semantics. However, it considers only propositional connectives, whereas universal and existential quantifiers are supported in LTN.

Inference in DLM requires maximizing the prediction of the model, which might be prohibitive in the presence of a large number of instances. In LTN, inference involves only a forward pass through the neural component which is rather simple and can be carried out in parallel. However, in DLM the weight associated with constraints can be learned, while in LTN they are specified in the background knowledge.

The approach taken in Knowledge Enhanced Neural Networks (KENN) [15] is similar to that of DLM. Starting from the predictions $\mathbf{y} = f_{nn}(\mathbf{x} \mid \mathbf{w})$ made by a base neural network $f_{nn}(\cdot \mid \mathbf{w})$, KENN adds a *knowledge enhancer*, which is a function that modifies $\mathbf{y}$ based on a set of weighted constraints formulated in terms of clauses. The formal model can be specified as follows:

$$\text{KENN}(\mathbf{x} \mid \boldsymbol{\lambda}, \mathbf{w}) = \sigma(f'_{nn}(\mathbf{x} \mid \mathbf{w}) + \sum_c \lambda_c \cdot (\text{softmax}(\text{sign}(c) \odot f'_{nn}(\mathbf{x} \mid \mathbf{w})) \odot \text{sign}(c)))$$

where $f'_{nn}(\mathbf{x} \mid \mathbf{w})$ are the pre-activations of $f_{nn}(\mathbf{x} \mid \mathbf{w})$, $\text{sign}(c)$ is a vector of the same dimension of $\mathbf{y}$ containing 1, -1 and $-\infty$, such that $\text{sign}(c)_i = 1$ (resp. $\text{sign}(c)_i = -1$) if the i -th atom occurs

positively (resp. negatively) in c , or $-\infty$ otherwise, and $\odot$ is the element-wise product. KENN learns the weights λ of the clauses in the background knowledge and the base network parameters w by minimizing some standard loss, (e.g. cross-entropy) on a set of training data. If the training data is inconsistent with the constraint, the weight of the constraint will be close to zero. This intuitively implies that the latent knowledge present in the data is preferred to the knowledge specified in the constraints. In LTN, instead, training data and logical constraints are represented uniformly with a formula, and we require that they are both satisfied. A second difference between KENN and LTN is the language: while LTN supports constraints written in full first-order logic, constraints in KENN are limited to universally quantified clauses.

Systems that add knowledge to a neural network by adding a term to the loss function:. In [33], a framework is proposed that learns simultaneously from labeled data and logical rules. The proposed architecture is made of a *student* network f_{nn} and a *teacher* network, denoted by q . The student network is trained to do the actual predictions, while the teacher network encodes the information of the logical rules. The transfer of information from the teacher to the student network is done by defining a joint loss $\mathcal{L}$ for both networks as a convex combination of the loss of the student and the teacher. If $\tilde{y} = f_{nn}(x | w)$ is the prediction of the student network for input x , the loss is defined as:

$$(1 - \pi) \cdot \mathcal{L}(y, \tilde{y}) + \pi \cdot \mathcal{L}(q(\tilde{y} | x), \tilde{y})$$

where $q(\tilde{y} | x) = \exp(-\sum_c \lambda_c(1 - \phi_c(x, \tilde{y})))$ measures how much the predictions $\tilde{y}$ satisfy the constraints encoded in the set of clauses $\{\lambda_c : \phi_c\}_{c \in C}$. Training is iterative. At every iteration, the parameters of the student network are optimized to minimize the loss that takes into account the feedback of the teacher network on the predictions from the previous step. The main difference between this approach and LTN is how the constraints are encoded in the loss. LTN integrates the constraints in the network and optimizes directly their satisfiability with no need for additional training data. Furthermore, the constraints proposed in [33] are universally quantified formulas only.

The approach adopted by LYRICS [47] is analogous to the first version of LTN [61]. Logical constraints are translated into a loss function that measures the (negative) satisfiability level of the network. Differently from LTN, formulas in LYRICS can be associated with weights that are hyper-parameters. In [47], a logarithmic loss function is also used when the product t-norm is adopted. Notice that weights can also be added (indirectly) to LTN by introducing a 0-ary predicate p_w to represent a constraint of the form $p_w \wedge \phi$. An advantage of this approach would be that the weights could be learned.

In [72], a neural network computes the probability of some events being true. The neural network should satisfy a set of propositional logic constraints on its output. These constraints are compiled into arithmetic circuits for weighted model counting, which are then used to compute a loss function. The loss function then captures how close the neural network is to satisfying the propositional logic constraints.

Systems that apply logical reasoning on the predictions of a base neural network:. The most notable architecture in this category is DeepProbLog [41]. DeepProbLog extends the ProbLog framework for probabilistic logic programming to allow the computation of probabilistic evidence from neural networks. A ProbLog program is a logic program where facts and rules can be associated with probability values. Such values can be learned. Inference in ProbLog to answer a query q is